A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval-Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE EIGHT

Agentic RAG with LangGraph

Beginner to Industry Level • Assumes only basic Python

MODULE EIGHT

Agentic RAG with LangGraph

Every RAG system so far has been a fixed pipeline: the same steps, in the same order, for every query. This module gives RAG the ability to think — to decide whether and how to retrieve, to grade its own results, to retry, to use tools, and to loop until the answer is good. You will master LangGraph, the framework for building such systems: its state, nodes, edges, conditional routing, checkpointing, and memory. You will learn the core agentic design patterns — ReAct, plan-and-execute, and reflection — and multi-agent orchestration. By the end you will turn the Corrective RAG concept from Module 7 into a running, self-correcting agentic system.

WHERE WE ARE

Module 7 ended with a recurring signal: the self-correcting patterns — CRAG and Self-RAG — need to *branch* and *loop*, which a linear LCEL chain cannot express. This module supplies the missing tool. LangGraph is how you build pipelines with decisions, cycles, and memory. Everything before this module built the *components* of RAG; this module builds the *brain* that orchestrates them intelligently.

8.1 Introduction

Pause and notice something about every RAG system in this book so far. No matter how advanced — hybrid search, re-ranking, RAG Fusion, HyDE — they have all shared one rigid property: **they are fixed pipelines.** The same sequence of steps runs for every single query. Embed, search, augment, generate. A trivial question and a fiendishly complex one travel the identical assembly line. The pipeline cannot decide to skip retrieval, cannot decide to retrieve a second time, cannot notice its own results were poor and try again, cannot pick between two knowledge sources. It does what it was wired to do, every time, regardless of the situation.

That rigidity is the ceiling we now break through. **Agentic RAG** replaces the fixed pipeline with a system that can *reason about its own process and decide what to do next.* An agentic RAG system can look at a query and decide retrieval is unnecessary. It can retrieve, examine what came back, judge it inadequate, reformulate the query, and retrieve again. It can choose between a vector store, a web search, and a database. It can

draft an answer, critique that draft against the evidence, and revise it. It can loop — doing whatever it takes, in whatever order, until the job is done.

This is a genuine shift in kind, not degree. A fixed pipeline is a *recipe*; an agent is a *cook*. And building a cook requires a different tool than building a recipe. LCEL chains, which you have used since Module 1, are excellent for recipes — fixed, linear sequences — but they fundamentally cannot express "loop until satisfied" or "if the results are bad, go do something else." Those require **cycles** and **conditional branches**, and that is precisely what **LangGraph** provides.

This module teaches both halves: the *concept* of agentic RAG, and the *tool*, LangGraph, for building it. We will dissect LangGraph's building blocks — state, nodes, edges, conditional routing, checkpointing, memory — study the standard agentic design patterns, cover multi-agent orchestration, and finish by building a complete, self-correcting agentic RAG system in running code.

8.2 Why This Topic Matters

> **WHY THIS MATTERS**
>
> Agentic RAG is where the field is heading, and it is where the most valuable engineering work now lives. Fixed-pipeline RAG is becoming a commodity; the systems companies actually want — assistants that reason, self-correct, use multiple tools, and handle genuinely hard queries reliably — are agentic. LangGraph is the industry-standard framework for building them. Mastering this module is the difference between being able to *assemble* a RAG pipeline and being able to *architect* an intelligent system. It is also the most heavily probed topic in senior AI-engineering interviews today.

Four concrete reasons this module repays close study:

1. **Fixed pipelines have a hard ceiling.** Some queries genuinely need iteration, tool choice, and self-correction. No amount of tuning a linear chain provides those; only an agentic architecture does.
2. **It makes Module 7's best patterns real.** CRAG and Self-RAG were taught as concepts because a chain cannot run them. LangGraph is the tool that turns them into working code.
3. **LangGraph is a foundational skill.** It is not RAG-specific — it is the standard way to build *any* stateful, multi-step, looping LLM application. The skill transfers everywhere.
4. **It is the production frontier.** Checkpointing, durable execution, human-in-the-loop, and memory — the features that make agents reliable enough to deploy — are all LangGraph concepts taught here.

8.3 A Beginner-Friendly Explanation

Let us separate two ideas: *what an agent is*, and *what LangGraph is*.

What is an agent? An ordinary program follows fixed instructions. An **agent** is a program where an **LLM is in the driver's seat, deciding what to do next.** Instead of you hard-coding "step 1, then step 2, then step 3," you give the LLM a *goal* and a set of *tools* (a retriever, a web search, a calculator), and the LLM repeatedly decides: *which tool should I use now? have I finished? should I try again?* The agent **thinks**, **acts** (uses a tool), **observes** the result, and **thinks again** — looping until the goal is met. The control flow is no longer fixed by you; it emerges from the LLM's reasoning.

Agentic RAG, then, is RAG where retrieval is not a mandatory first step but a **tool the agent chooses to use.** The agent decides whether to retrieve, what query to retrieve with, whether to retrieve again, and whether the retrieved evidence is good enough — exactly the decisions that Module 7's CRAG and Self-RAG patterns described.

What is LangGraph? If you let an LLM loop and branch freely, you need a structured, reliable way to manage that flow — otherwise it becomes an untraceable tangle. **LangGraph** is a framework for building applications as a **graph**: a flowchart of steps. You define the steps (**nodes**), the arrows between them (**edges**), the decision points (**conditional edges**), and a shared notepad of information that travels through the whole flow (the **state**). Because a graph can contain *loops* and *branches* — unlike a straight-line chain — it can express exactly the "think, act, observe, repeat" behaviour an agent needs.

So the plain-language summary: an **agent** is an LLM that decides its own steps; **agentic RAG** makes retrieval one of those decided steps; and **LangGraph** is the flowchart-with-a-notepad framework you use to build it reliably.

> **DEFINITION**
>
> An **agent** is a system in which an LLM dynamically decides the sequence of actions to take to accomplish a goal, typically by reasoning, using tools, observing results, and looping. **Agentic RAG** is RAG in which retrieval is one such tool the agent chooses to use, when and how it judges appropriate. **LangGraph** is a framework for building agentic and other stateful applications as a graph of nodes (steps), edges (transitions), and a shared state, supporting cycles and conditional branching.

8.4 Real-World Analogy: The Recipe vs the Chef

REAL-WORLD ANALOGY

A fixed RAG pipeline (everything before this module) is a **recipe card**. It lists exact steps in exact order: chop, mix, bake 30 minutes, serve. Follow it precisely and you get a predictable result — but the recipe cannot adapt. If the oven runs hot, the recipe still says "bake 30 minutes." If an ingredient is missing, the recipe has no answer. It cannot taste the dish and decide it needs more salt.

An agentic RAG system is a **chef**. Given the goal "make a good meal," the chef *decides*: taste, adjust, taste again, substitute a missing ingredient, check the oven, and loop until the dish is right. The chef reasons, acts, observes, and repeats.

LangGraph is the professional kitchen that lets the chef work reliably. Each **station** (prep, stove, plating) is a *node*. The **paths** between stations are *edges*. A **decision** — "is it seasoned enough? if not, back to the stove" — is a *conditional edge*. The **order ticket** that travels with the dish, gathering notes at each station, is the *state*. And **writing the ticket's progress down** so a shift change does not lose the order is *checkpointing*.

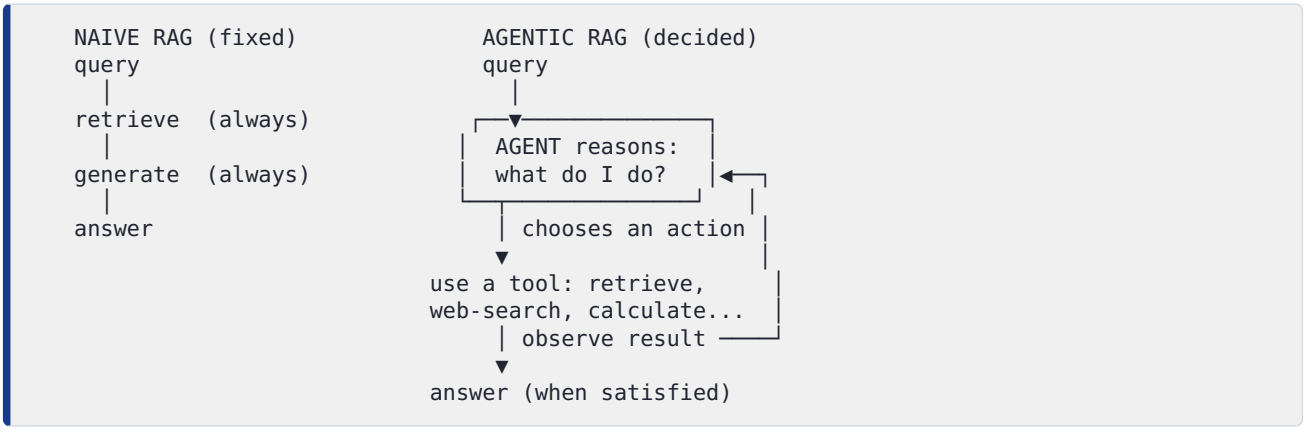
This analogy carries the whole module:

In the analogy	In LangGraph
A kitchen station (prep, stove, plating)	A **node** — one step of work
A path from one station to the next	An **edge** — a transition
"If under-seasoned, return to the stove"	A **conditional edge** — routing by a decision
The order ticket carrying notes through the kitchen	The **state** — shared data passed through the graph
Writing down progress so a shift change loses nothing	**Checkpointing** — durable, resumable execution
The chef tasting and adjusting	The **agentic loop** — reason, act, observe, repeat

8.5 Core Concepts: Agentic RAG and RAG as a Tool

Before LangGraph mechanics, fix the conceptual shift.

Naive RAG vs agentic RAG. Naive RAG is *control flow you wrote*: retrieve, then generate, always. Agentic RAG is *control flow the LLM decides*: the agent is handed tools and a goal and works out the steps itself.



RAG as a tool. In an agentic system, the retriever is wrapped as a **tool** — a function the agent may call, with a name and a description telling the agent *when* it is useful (e.g. "*search_company_docs: use this to answer questions about company policies*"). The agent reads the user's request, reads its tools' descriptions, and decides whether and which tool to call. Retrieval becomes a *capability the agent reaches for*, not a mandatory first step. This is why agentic RAG can correctly skip retrieval for "*hello*" and correctly retrieve twice for a hard multi-part question — the decision is the LLM's, made fresh each time.

Why this needs a new tool. Agentic behaviour has two properties an LCEL chain structurally cannot provide:

- **Cycles (loops).** "Think, act, observe, think again" is a loop. Chains are *directed acyclic* — no cycles allowed by design.
- **Conditional branching.** "If the evidence is weak, reformulate and retry; otherwise generate" is an if/else. Chains run a fixed sequence with no branch points.

LangGraph exists precisely to add cycles and conditional branching to LLM applications. That is the entire reason it is a *graph* and not a *chain*.

8.6 LangGraph Fundamentals

A LangGraph application has four ingredients: a **state**, **nodes**, **edges**, and a **compiled graph**. We will take each in turn, building toward the code in Section 8.14.

8.6.1 State — The Shared Notepad

The **state** is a single data structure that travels through the entire graph. Every node reads from it and writes updates back to it. It is the order ticket, the shared notepad — the one place all information lives as the workflow runs.

In LangGraph you define the state's shape with a `TypedDict` — a dictionary with declared, named fields.

```python
from typing import TypedDict, List, Annotated
from operator import add
from langchain_core.documents import Document

class RAGState(TypedDict):
    question: str                              # the user's question
    documents: List[Document]                  # chunks retrieved so far
    generation: str                            # the drafted answer
    retries: int                               # loop-safety counter
    messages: Annotated[list, add]             # conversation, APPENDED to
```

One subtlety matters. By default, when a node returns a value for a state field, it **overwrites** that field. Sometimes you want to **accumulate** instead — for a conversation history, each new message should be *appended*, not replace the whole history. You express that with a **reducer**: `Annotated[list, add]` tells LangGraph "when a node returns a value for `messages`, *add* it to the existing list rather than replace it." Overwrite is the default; reducers customise it. This distinction — overwrite versus accumulate — is a frequent source of confusion, so fix it now.

8.6.2 Nodes — The Workstations

A **node** is one step of work. In code, a node is simply a **Python function** that takes the current state and returns a **partial update** to it — a dictionary of just the fields it wishes to change.

```python
def retrieve(state: RAGState) -> dict:
    """A node: fetch chunks for the current question."""
    question = state["question"]
    docs = retriever.invoke(question)
    return {"documents": docs}      # update ONLY the 'documents' field
```

Note the discipline: the node reads what it needs from `state`, does its work, and returns *only the fields it changed*. LangGraph merges that partial update into the running state (overwriting, or applying a reducer). A node can do anything — call an LLM, call a retriever, call a tool, run plain Python.

8.6.3 Edges — The Paths

Edges connect nodes and define the flow. There are two kinds, and the difference is the most important idea in LangGraph.

A **normal edge** is an unconditional path: *after node A, always go to node B.*

```python
builder.add_edge("retrieve", "grade_documents")   # retrieve → ALWAYS → grade
```

A **conditional edge** is a *decision*. Instead of a fixed destination, you supply a **router function** that inspects the state and *returns the name of the next node*. This is how a graph branches — and, by routing back to an earlier node, how it loops.

```python
def decide_next_step(state: RAGState) -> str:
    """Router: choose the next node based on the state."""
    if state["documents"]:                # relevant chunks were found
        return "generate"
    elif state["retries"] < 3:            # none found, but retries left
        return "rewrite_query"            # → loops back to retry
    else:
        return "give_up"                  # exhausted retries

builder.add_conditional_edges(
    "grade_documents",      # after this node...
    decide_next_step,       # ...run this router to choose the destination
    {                       # map the router's return value → a node name
        "generate": "generate",
        "rewrite_query": "rewrite_query",
        "give_up": "give_up",
    },
)
```

Conditional edges are where agentic behaviour *lives*. The branch ("good chunks → generate; bad chunks → rewrite") is the system *deciding*. And because `rewrite_query` can lead back to `retrieve`, the graph contains a **cycle** — the self-correction loop. This is the exact capability a chain lacks.

Two special nodes bound the graph: `START`, the entry point, and `END`, the exit. Every graph has an edge from `START` and at least one path to `END`.

8.6.4 Compiling and Running the Graph

You assemble the graph with a `StateGraph`, then **compile** it into a runnable object.

```python
from langgraph.graph import StateGraph, START, END

builder = StateGraph(RAGState)            # a graph over our state shape
builder.add_node("retrieve", retrieve)    # register each node
builder.add_node("grade_documents", grade_documents)
builder.add_node("generate", generate)
builder.add_node("rewrite_query", rewrite_query)

builder.add_edge(START, "retrieve")       # entry point
builder.add_edge("retrieve", "grade_documents")
builder.add_conditional_edges("grade_documents", decide_next_step, {...})
builder.add_edge("rewrite_query", "retrieve")   # the loop edge
builder.add_edge("generate", END)

graph = builder.compile()                 # → a runnable application
result = graph.invoke({"question": "How much annual leave do I get?",
                       "retries": 0})
```

The compiled `graph` exposes the same `.invoke()` interface as a chain — so an agentic graph can itself be used as a component inside other LangChain code.

WHY THIS MATTERS

The whole power of LangGraph reduces to one idea: **nodes do work; conditional edges decide.** A linear chain is all "do work" and no "decide." By adding conditional edges — routers that pick the next node from the current state — LangGraph lets a system branch and loop, which is the structural prerequisite for an agent. When you read or design a LangGraph application, trace the *conditional edges* first: they are the decision points, and the decision points are where the intelligence lives.

8.7 Checkpointing and Durable Execution

A compiled graph runs fine on its own, but production agents need one more capability: **memory of where they are.** This is provided by a **checkpointer**.

When you compile a graph *with a checkpointer*, LangGraph **saves the entire state after every node executes**. Each saved snapshot is a **checkpoint**. This single mechanism unlocks four production-critical features:

- **Durable execution.** If the process crashes mid-run — or a long workflow is paused waiting for something — execution can **resume from the last checkpoint** instead of starting over. Hours of agent work, or a half-finished multi-step task, are not lost.
- **Human-in-the-loop.** Because the graph can pause and its state is saved, you can interrupt it before a sensitive node (say, one that sends an email or spends money), let a human inspect and approve, then resume. The checkpoint is what makes "pause for approval" possible.
- **Memory (conversation history).** Checkpoints are grouped by a `thread_id`. All the runs sharing one `thread_id` form one continuous conversation — so the agent remembers earlier turns. This is short-term memory, and it is covered next.
- **Time travel.** You can rewind to an earlier checkpoint and re-run from there with different inputs — invaluable for debugging an agent's decisions.

```python
from langgraph.checkpoint.memory import MemorySaver

# Compile WITH a checkpointer → state is saved after every node
graph = builder.compile(checkpointer=MemorySaver())

# A thread_id ties runs together into one conversation
config = {"configurable": {"thread_id": "user-42-session-1"}}
graph.invoke({"question": "How much annual leave?"}, config=config)
graph.invoke({"question": "And for part-time staff?"}, config=config)
# ↑ the second call REMEMBERS the first, because the thread_id matches
```

`MemorySaver` keeps checkpoints in memory — fine for development. Production uses a *persistent* checkpointer (backed by a database such as PostgreSQL) so durability and memory survive a full restart.

DEFINITION

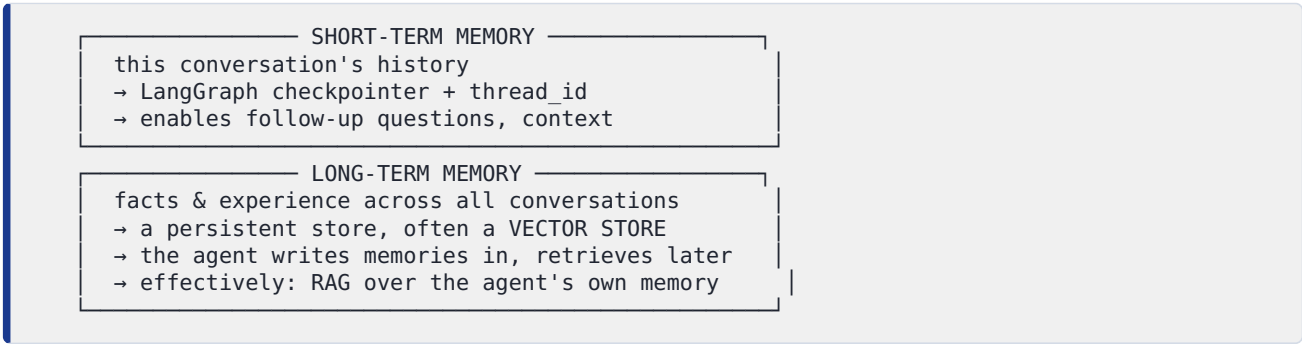
Checkpointing is LangGraph's mechanism of saving the graph's complete state after every node. **Durable execution** is the resulting ability of a workflow to survive interruptions — crashes, restarts, or deliberate pauses — and resume exactly where it left off, because its progress was checkpointed.

8.8 Memory Systems

Agents need to *remember*, and there are two distinct kinds of memory — do not conflate them.

Short-term memory (within a conversation). This is the history of the current conversation: earlier questions and answers in this session. In LangGraph it is provided directly by **checkpointing + a** `thread_id`, as just shown. All runs on the same thread share state, so the agent naturally sees what was said earlier in that thread. This is what makes a follow-up like "*and for part-time staff?*" work — and it is also why query rewriting (Module 6) needs the conversation history that short-term memory holds.

Long-term memory (across conversations). This is information that should persist *beyond* a single conversation — facts about a user learned weeks ago, knowledge accumulated across many sessions, an agent's own past conclusions. Thread-scoped short-term memory cannot do this, because a new conversation is a new thread. Long-term memory is instead stored in a separate **store** — and very often that store is a **vector store** (Module 4): the agent writes memories into it as embedded text and *retrieves* relevant memories later by similarity search. In other words, **long-term agent memory is itself a RAG system** — retrieval over the agent's own remembered experience.



INTERVIEW INSIGHT

"How do you give an agent memory?" expects you to distinguish the two kinds. **Short-term** memory is conversation history within a session — in LangGraph, provided by a checkpointer keyed on a `thread_id`. **Long-term** memory persists across sessions and is stored separately, commonly in a vector store the agent writes to and retrieves from by similarity — which means long-term memory is, architecturally, RAG applied to the agent's own past. Naming both, and noting that long-term memory reuses the vector-store machinery from Module 4, signals real depth.

8.9 Agentic RAG Design Patterns

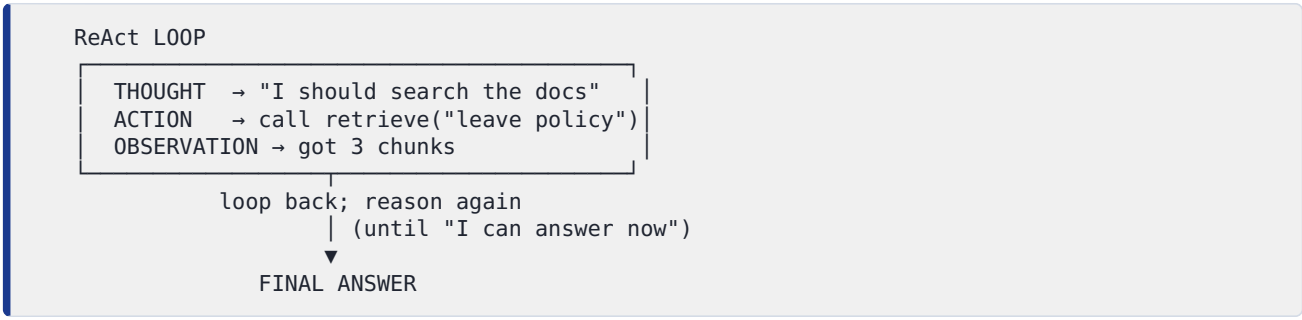
There are three canonical patterns for structuring an agent's reasoning. Each is a different shape of graph.

8.9.1 ReAct — Reason + Act

ReAct (Reasoning and Acting) is the foundational agentic pattern. The agent runs a loop of three repeating steps:

1. **Thought** — the LLM reasons about what to do next given the goal and what it knows so far.
2. **Action** — it takes an action, typically calling a tool (retrieve, web-search, calculate).
3. **Observation** — it receives the tool's result and feeds it back into the next Thought.

The loop repeats — *Thought* → *Action* → *Observation* → *Thought* → *...* — until the agent reasons that it has enough to answer, at which point it produces the final response.



In LangGraph, ReAct is a graph with an *agent* node (the Thought) and a *tools* node (the Action), connected by a conditional edge: after the agent node, route to *tools* if the agent chose a tool, or to `END` if it produced an answer; after *tools*, always route back to the agent. The cycle *is* the ReAct loop. ReAct is flexible and general-purpose — the default choice for a single-agent system.

8.9.2 Plan-and-Execute

ReAct decides one step at a time, which can wander on complex multi-step tasks. **Plan-and-execute** adds structure: it **plans the whole task upfront, then executes the plan.**

1. A **planner** node uses the LLM to break the goal into an explicit, ordered list of sub-steps.
2. An **executor** carries out the steps one by one (each step may itself use tools, even a ReAct loop).
3. A **re-planner** checks progress after each step and revises the remaining plan if needed.

Plan-and-execute is better for complex, multi-stage tasks because the explicit upfront plan keeps the agent goal-directed instead of improvising every move. Its cost is rigidity and the extra planning call.

8.9.3 Reflection

The **reflection** pattern adds *self-criticism*. The agent produces output, then a separate **critic** step evaluates that output and generates feedback, and the agent **revises** — looping until the critic is satisfied (or a retry limit is hit).

```
GENERATE  →  CRITIQUE  →  satisfied? ──yes──▶ done
    ▲                          │
    └──── revise ──────┘   (no: feed critique back, regenerate)
```

You have already met reflection in disguise: **Self-RAG (Module 7) is the reflection pattern applied to RAG** — the faithfulness and usefulness checks are the critic; the loop-back to regenerate is the revision. Reflection trades extra LLM calls for substantially higher output quality, and is worth it for high-stakes answers.

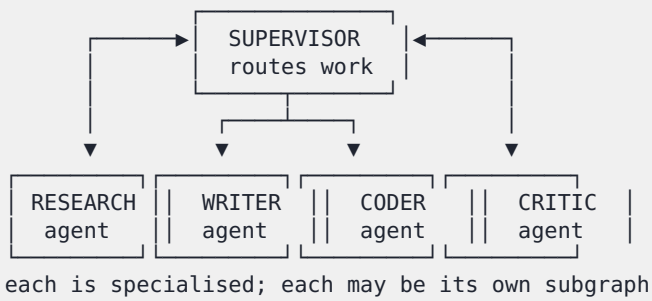
> **BEST PRACTICE**
>
> Match the pattern to the task. For a general assistant that picks among a few tools, **ReAct** is the right default — simple and flexible. For complex, genuinely multi-stage tasks, **plan-and-execute** keeps the agent on track. When answer *quality* is paramount and you can afford extra LLM calls, add a **reflection** loop. And note these compose: a real system might plan-and-execute at the top level, run a ReAct loop inside each step, and wrap the final answer in a reflection check. Start with the simplest pattern that works, and add structure only when a task demands it.

8.10 Multi-Agent Orchestration

When a task is large and varied, a single agent juggling many tools and responsibilities becomes confused and unreliable — its prompt grows unwieldy, and it makes worse

decisions. The solution is **multi-agent orchestration**: divide the work among several *specialised* agents, each focused and good at one thing, coordinated into a team.

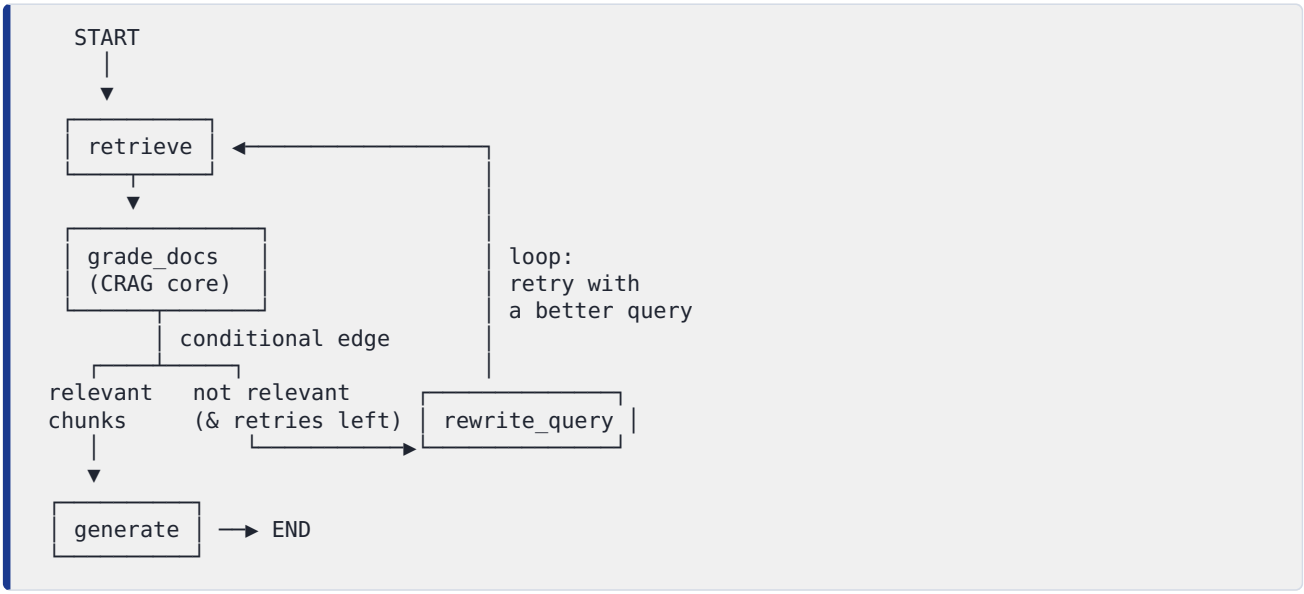
The most common coordination structure is the **supervisor pattern**: a *supervisor* agent receives the task and **routes** it to the appropriate *worker* agent (a researcher, a writer, a coder, a critic), collects the result, and decides what to do next — route to another worker, or finish.



LangGraph implements this elegantly: **each agent can itself be a graph**, and a graph can be used as a node inside another graph. So the supervisor is a graph whose nodes are themselves agent-graphs. Other structures exist — agents arranged as a *network* that can hand off to each other directly, or a *hierarchy* of supervisors — but the supervisor pattern is the standard, reliable starting point. The benefits are *separation of concerns* (each agent's prompt and tools stay focused), *easier debugging* (you can test one agent in isolation), and *scalability* (add a capability by adding an agent).

8.11 Building Agentic RAG: CRAG as a Graph

We can now keep Module 7's promise. **Corrective RAG**, described there as a concept impossible to build with a chain, is a natural LangGraph application. Here is its graph shape:



Every Module 7 idea maps onto a LangGraph construct: the *grade* step is a node; the *correct / incorrect* verdict is a **conditional edge**; the *fallback / retry* is the **loop edge** back to retrieval; the *retry limit* is a counter in the **state**. The concept becomes code because LangGraph supplies branching and cycles. The code in Section 8.14 implements exactly this graph.

8.12 Production-Grade Agent Systems

Moving an agent from a notebook to production adds concerns beyond the graph logic:

- **Loop safety.** An agent that loops can loop *forever* if a condition never resolves. Every cycle needs a guard — a retry counter in the state, a recursion limit on the graph — so a misbehaving agent terminates rather than running up an unbounded bill.
- **Cost and latency control.** Agents make *many* LLM calls — every Thought, every grade, every reflection. Costs and response times must be budgeted, monitored, and capped.
- **Durable execution and persistence.** A persistent (database-backed) checkpointer so long-running or paused workflows survive restarts.
- **Human-in-the-loop.** For consequential actions (spending money, sending communications, modifying data), pause for human approval before the sensitive node.
- **Observability and tracing.** An agent's path is *dynamic* — you cannot debug it without seeing the actual sequence of nodes, decisions, and tool calls it took. Tracing (LangSmith, covered in Module 10) is essential, not optional.
- **Tool reliability and error handling.** Tools fail — a web search times out, an API errors. Nodes must handle tool failures gracefully and the graph must route around them rather than crashing.

- **Guardrails.** Validate inputs and outputs; constrain what tools the agent may call and with what arguments (security and guardrails are detailed in the Production RAG section).

> **COMMON MISTAKE**
>
> Shipping an agent with an unbounded loop. It is the single most dangerous agentic bug. A conditional edge that routes "retry" whenever results are imperfect — with no counter and no limit — can loop indefinitely on a hard query, making LLM calls forever and running up a real, large bill before anyone notices. **Every loop in an agentic graph must have an explicit termination guard:** a retry counter checked in the routing function, and a graph-level recursion limit as a backstop. Build the guard in from the very first version, not after the first runaway bill.

8.13 Workflow

The mental model for building agentic RAG:

1. **Decide if you even need an agent.** If a fixed pipeline handles your queries well, use one — it is simpler, cheaper, and more predictable. Reach for agentic RAG only when queries genuinely need iteration, tool choice, or self-correction.
2. **Design the state.** What information must travel through the workflow? Define the `TypedDict`, and decide which fields overwrite and which accumulate (reducers).
3. **Identify the nodes.** Each distinct step of work — retrieve, grade, generate, rewrite — is a node function.
4. **Identify the decisions.** Each branch or loop is a conditional edge with a router function. These are where the agentic intelligence lives.
5. **Add loop guards.** Every cycle gets a retry counter and a termination condition.
6. **Add checkpointing** for memory and durability, with a `thread_id` per conversation.
7. **Compile, trace, and test** — exercise the hard paths (loops, fallbacks), and use tracing to see the actual route taken.

8.14 Step-by-Step Implementation and Code

We will build a complete, self-correcting **agentic RAG** system — the CRAG graph from Section 8.11. It retrieves, grades its own results, and if they are inadequate it rewrites the query and retries, with a strict loop guard. Save this as `module8_agentic_rag.py`.

```
"""
module8_agentic_rag.py
A self-correcting agentic RAG system built with LangGraph (CRAG pattern).
It retrieves, grades the results, and loops to retry with a better query.
Stack: Python + LangChain + LangGraph + Gemini + FAISS.
Install: pip install langgraph langchain langchain-community \
         langchain-google-genai faiss-cpu python-dotenv
"""

import os
from typing import TypedDict, List
from dotenv import load_dotenv
from langchain_google_genai import (
    GoogleGenerativeAIEmbeddings, ChatGoogleGenerativeAI)
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

docs = [
    Document(page_content="Full-time staff receive 24 days of paid annual "
             "leave per calendar year, accrued monthly."),
    Document(page_content="Sick leave covers up to 10 paid days per year, "
             "with a medical certificate required after 3 consecutive days."),
    Document(page_content="The office observes all national public holidays."),
]
vector_store = FAISS.from_documents(docs, embeddings)
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# ---------- 1. THE STATE: the shared notepad ----------
class RAGState(TypedDict):
    question: str            # current (possibly rewritten) search question
    original: str            # the user's original question
    documents: List[Document]  # chunks judged relevant
    generation: str          # the drafted answer
    retries: int             # loop-safety counter

# ---------- 2. THE NODES: units of work ----------
grader = (ChatPromptTemplate.from_template(
    "Is this document relevant to the question? Reply only 'yes' or 'no'.\n"
    "Question: {q}\nDocument: {d}") | llm | StrOutputParser())

answer_chain = (ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context. If the context is empty, "
    "say you don't have that information.\n\nContext:\n{ctx}\n\n"
    "Question: {q}") | llm | StrOutputParser())

rewrite_chain = (ChatPromptTemplate.from_template(
    "The search for this question returned nothing useful. Rewrite it as a "
    "clearer, keyword-rich search query. Return ONLY the query.\n\n"
    "Question: {q}") | llm | StrOutputParser())

def retrieve(state: RAGState) -> dict:
    """Node: fetch chunks for the current question."""
    print(f"  [retrieve] searching: '{state['question']}'")
    return {"documents": retriever.invoke(state["question"])}

def grade_documents(state: RAGState) -> dict:
    """Node (CRAG core): keep only chunks graded relevant."""
    relevant = []
    for d in state["documents"]:
        verdict = grader.invoke({"q": state["original"],
```

```
                                    "d": d.page_content}).strip().lower()
        if "yes" in verdict:
            relevant.append(d)
    print(f"  [grade] {len(relevant)}/{len(state['documents'])} relevant")
    return {"documents": relevant}

def rewrite_query(state: RAGState) -> dict:
    """Node: reformulate the query and increment the retry counter."""
    new_q = rewrite_chain.invoke({"q": state["original"]}).strip()
    print(f"  [rewrite] new query: '{new_q}'")
    return {"question": new_q, "retries": state["retries"] + 1}

def generate(state: RAGState) -> dict:
    """Node: write the grounded answer from the relevant chunks."""
    ctx = "\n\n".join(d.page_content for d in state["documents"])
    return {"generation": answer_chain.invoke(
        {"ctx": ctx, "q": state["original"]})}

# ---------- 3. THE ROUTER: a conditional-edge decision ----------
def decide_after_grading(state: RAGState) -> str:
    """If relevant chunks were found -> generate. Else retry, with a guard."""
    if state["documents"]:
        return "generate"
    if state["retries"] < 2:          # LOOP GUARD: at most 2 retries
        return "rewrite_query"
    return "generate"                 # give up retrying; generate honestly

# ---------- 4. ASSEMBLE AND COMPILE THE GRAPH ----------
builder = StateGraph(RAGState)
builder.add_node("retrieve", retrieve)
builder.add_node("grade_documents", grade_documents)
builder.add_node("rewrite_query", rewrite_query)
builder.add_node("generate", generate)

builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "grade_documents")
builder.add_conditional_edges(
    "grade_documents", decide_after_grading,
    {"generate": "generate", "rewrite_query": "rewrite_query"},
)
builder.add_edge("rewrite_query", "retrieve")     # the self-correction loop
builder.add_edge("generate", END)

graph = builder.compile(checkpointer=MemorySaver())   # checkpointer = memory

# ---------- 5. RUN IT ----------
if __name__ == "__main__":
    config = {"configurable": {"thread_id": "demo-session-1"}}
    for q in ["How many paid annual leave days do full-time staff get?",
              "What is the company's policy on cryptocurrency bonuses?"]:
        print(f"\nQUESTION: {q}")
        result = graph.invoke(
            {"question": q, "original": q, "documents": [], "retries": 0},
            config=config,
        )
        print(f"ANSWER: {result['generation']}")
```

8.15 Line-by-Line Code Explanation

Setup. The Gemini embedding and chat models are created once; `temperature=0` keeps grading and routing decisions deterministic. A small FAISS store and retriever are built from three documents — note there is *nothing* about cryptocurrency bonuses,

deliberately, so the second test query exercises the self-correction loop and the honest "I don't have that" path.

1 — the state. `RAGState` is the `TypedDict` that travels through every node. `question` is the *current* search query (it changes when rewritten); `original` preserves the user's true question so grading and answering always judge against the real intent; `documents` holds the relevant chunks; `generation` holds the answer; `retries` is the all-important loop guard. Every field here uses default overwrite semantics — no reducer is needed for this graph.

2 — the nodes. Three small LCEL chains (`grader`, `answer_chain`, `rewrite_chain`) are the LLM-powered pieces. Then four node functions, each taking the state and returning a *partial update*: `retrieve` searches with the current `question` and returns the chunks; `grade_documents` is the CRAG core — it runs the grader on each chunk and returns only the relevant ones; `rewrite_query` reformulates the query *and* increments `retries`; `generate` builds the context from the relevant chunks and writes the grounded answer. Each node prints a trace line so you can watch the agent's path at runtime.

3 — the router. `decide_after_grading` is the conditional-edge function — the decision point, and the heart of the agentic behaviour. Its logic: if relevant chunks were found, go to `generate`; if none were found *and* fewer than 2 retries have been used, go to `rewrite_query` (which loops back); if retries are exhausted, go to `generate` anyway so the system answers honestly ("I don't have that information") rather than looping forever. **That retry check is the loop guard** — without it, an unanswerable query would cycle endlessly.

4 — assemble and compile. The four nodes are registered. The edges wire the flow: `START → retrieve → grade_documents`, then a **conditional edge** out of `grade_documents` using the router, a normal edge `rewrite_query → retrieve` (this is the **cycle** — the self-correction loop), and `generate → END`. `builder.compile(checkpointer=MemorySaver())` produces the runnable graph *with* checkpointing, so state is saved after every node and the `thread_id` gives the system memory.

5 — run it. Two queries are invoked on the same `thread_id`. The first — about annual leave — is in the knowledge base: `retrieve` finds it, `grade_documents` keeps it, the router sends it straight to `generate`. The second — about cryptocurrency bonuses — is *not* in the knowledge base: `grade_documents` finds zero relevant chunks, the router triggers `rewrite_query`, the graph loops back to `retrieve` and tries again, and after the retry limit it routes to `generate`, which honestly states the information is unavailable. Running the file and watching the printed trace lets you *see* the agent retrieve, grade, decide, loop, and finally answer.

BEST PRACTICE

Notice how readable this agent is precisely *because* it is a graph. Each node is a small, independently testable function. The control flow lives in one place — the router and the edges. To change the agent's behaviour (add a web-search fallback, add a reflection check on the answer) you add a node and an edge, without touching the others. This is why LangGraph scales to complex agents where tangled `if/else` code would not: the structure stays explicit, inspectable, and modular.

8.16 Common Mistakes

- **Using an agent when a fixed pipeline would do.** Agents are more complex, slower, and costlier. Only go agentic when queries genuinely need iteration, tool choice, or self-correction.
- **Unbounded loops.** A retry edge with no counter and no limit can loop forever, burning money. Every cycle needs an explicit termination guard.
- **Forgetting reducers.** Returning a value for a field that should *accumulate* (like conversation history) without an `Annotated[..., add]` reducer silently overwrites it. Know which fields overwrite and which accumulate.
- **Confusing the two kinds of memory.** Short-term (conversation, via checkpointer + `thread_id`) and long-term (cross-session, via a store) are different mechanisms for different needs.
- **Trying to build agentic flows with LCEL chains.** Chains cannot loop or branch. Branching, looping logic belongs in a LangGraph graph.
- **No observability.** An agent's path is dynamic; without tracing you cannot debug why it took the route it did.
- **Overstuffing a single agent.** One agent with a dozen tools and responsibilities makes poor decisions. Split it into specialised agents under a supervisor.
- **No human-in-the-loop on consequential actions.** An autonomous agent that can spend money or send messages without an approval checkpoint is a production risk.

8.17 Best Practices

- **Default to the simplest design** — a fixed pipeline if it suffices, then ReAct, then plan-and-execute or reflection only as task complexity demands.
- **Guard every loop** with a retry counter and a graph recursion limit.
- **Design the state deliberately**, choosing overwrite vs accumulate (reducers) for each field.
- **Keep nodes small and single-purpose** so they are testable and replaceable.

- **Put the intelligence in conditional edges** and keep routing logic clear and centralised.
- **Use checkpointing** for memory and durable execution; a persistent checkpointer in production.
- **Distinguish short-term and long-term memory**, using a vector store for the latter.
- **Add tracing, error handling, cost limits, and human-in-the-loop** before going to production.

8.18 Real-World Applications

- **Self-correcting knowledge assistants** — agentic RAG that grades its own retrievals and retries or falls back, instead of answering from poor evidence.
- **Research agents** — agents that plan a multi-step investigation, retrieve from several sources, synthesise, and reflect on the result (a direct basis for the *Research Assistant* project).
- **Customer-support agents** — agents that decide whether to search the knowledge base, escalate, or ask a clarifying question, and remember the conversation across turns.
- **Multi-agent document workflows** — a supervisor coordinating researcher, analyst, and writer agents to produce a report from a document corpus.
- **Coding assistants** — ReAct agents that retrieve from documentation, run code, observe errors, and iterate.
- **Long-running autonomous workflows** — durable, checkpointed agents that pause for human approval at sensitive steps and resume afterward.
- **Personalised assistants** — agents with long-term memory in a vector store, recalling user preferences and history across sessions.

8.19 Mini Project: An Agentic RAG Assistant with Self-Correction

Build a complete agentic RAG system and extend it:

1. Start from `module8_agentic_rag.py` and a knowledge base of your own (use Module 2's `ingest.py`).
2. Run it and confirm the two behaviours: an in-scope question routes straight to generation; an out-of-scope question triggers the rewrite-and-retry loop and then an honest "I don't have that" answer.

3. **Add a reflection step.** Insert a new node *after* `generate` that grades whether the answer is faithful to the retrieved chunks (Self-RAG style). If it is not, loop back to `generate` — with a retry guard. Trace a query through the new loop.
4. **Add a web-search fallback.** Change the router so that when retrieval fails after the retry limit, instead of giving up it routes to a `web_search` node (you may stub this) before generating — true CRAG behaviour.
5. **Add conversation memory.** Using the checkpointer and a fixed `thread_id`, ask a follow-up question that depends on a previous turn. Add a query-rewriting node (Module 6) that uses the conversation history so the follow-up resolves correctly.
6. **Add loop safety you can see.** Deliberately set the retry limit very high and confirm the recursion limit still terminates a genuinely unanswerable query. Then restore a sane limit.
7. **Trace and write up.** Print the full node-by-node path for three different queries. Write a short explanation of which conditional edges fired and why — demonstrating that you can read an agent's behaviour from its graph.

This mini project is a working, self-correcting agentic RAG system — and the architectural core of the Module 10 capstone.

8.20 Chapter Summary

- A fixed RAG pipeline runs the same steps for every query. **Agentic RAG** lets an **LLM decide its own steps** — whether to retrieve, with what query, whether to retry, which tool to use, whether to revise — looping until the goal is met.
- An **agent** reasons, acts (uses tools), observes, and repeats. In agentic RAG, **retrieval is a tool** the agent chooses, not a mandatory step.
- Agentic behaviour needs **cycles** (loops) and **conditional branching**, which linear LCEL chains cannot express. **LangGraph** adds exactly these.
- A LangGraph application has: a **state** (a shared `TypedDict` notepad; fields overwrite by default, or accumulate via a **reducer**), **nodes** (functions doing one step of work, returning partial state updates), **edges** (normal = always; **conditional** = a router function picks the next node — this is where branching and looping live), and a **compiled graph** bounded by `START` and `END`.
- **Checkpointing** saves the full state after every node, enabling **durable execution** (resume after crashes/pauses), **human-in-the-loop**, **memory**, and time travel.
- Memory has two kinds: **short-term** (conversation history, via checkpointer + `thread_id`) and **long-term** (cross-session, stored separately — often in a vector store, making long-term memory itself a RAG system).

- The three agentic design patterns: **ReAct** (reason → act → observe loop; the flexible default), **plan-and-execute** (plan the whole task, then execute; for complex multi-stage tasks), and **reflection** (generate → critique → revise loop; Self-RAG is reflection applied to RAG).
- **Multi-agent orchestration** divides large tasks among specialised agents, commonly under a **supervisor** that routes work; in LangGraph each agent can itself be a graph.
- Production agents add **loop guards** (mandatory — unbounded loops are the most dangerous bug), cost and latency limits, persistent checkpointing, human-in-the-loop on consequential actions, observability/tracing, and tool error handling.
- You built a complete self-correcting agentic RAG system (the CRAG pattern from Module 7) as a LangGraph graph with a self-correction loop and a retry guard.

8.21 Practice Exercises

1. **Conceptual.** Explain, in under 120 words, the difference between a fixed RAG pipeline and an agentic RAG system, and why agentic RAG needs LangGraph rather than an LCEL chain.
2. **State design.** Design the state `TypedDict` for an agentic research assistant that retrieves from documents and the web, drafts an answer, and reflects on it. List each field, its type, and whether it overwrites or accumulates.
3. **Edges.** Explain the difference between a normal edge and a conditional edge, and explain how a conditional edge enables both branching and looping.
4. **Memory.** A user returns to your assistant a week later and expects it to remember a preference they stated. Which kind of memory is needed, why short-term memory cannot do it, and how would you implement it?
5. **Patterns.** For each task, choose ReAct, plan-and-execute, or reflection and justify it: (a) a general assistant choosing among three tools; (b) writing a long report requiring research, analysis, and drafting; (c) producing a legally sensitive answer that must be verified before delivery.
6. **Code.** Run `module8_agentic_rag.py`. Trace the printed path for both queries. Then add a third query that is partially answerable and describe the route it takes.
7. **Loop safety.** In the code, explain exactly what would happen if `decide_after_grading` had no `retries` check, and why. Then describe two independent mechanisms that prevent runaway loops.

8.22 Interview Questions

BEGINNER

Q1. What is an agent?
A system in which an LLM dynamically decides the sequence of actions to take to reach a goal — typically by reasoning, using tools, observing results, and looping — rather than following a fixed, pre-written sequence of steps.

Q2. What is agentic RAG?
RAG in which retrieval is a tool the agent chooses to use, when and how it judges appropriate, rather than a mandatory first step. The agent can decide not to retrieve, to retrieve again, or to use a different source.

Q3. Why use LangGraph instead of an LCEL chain for an agent?
Agentic behaviour requires cycles (loops) and conditional branching. LCEL chains are linear and acyclic and cannot express loops or branches. LangGraph builds applications as a graph, which supports both.

Q4. What are the four main building blocks of a LangGraph application?
The state (a shared data structure passed through the graph), nodes (functions performing a step of work), edges (transitions between nodes, normal or conditional), and the compiled graph itself, bounded by START and END.

INTERMEDIATE

Q5. What is the difference between a normal edge and a conditional edge?
A normal edge always routes from one node to a fixed next node. A conditional edge runs a router function that inspects the state and returns the name of the next node, so the path can branch — and, by routing back to an earlier node, loop. Conditional edges are where an agent's decisions and cycles live.

Q6. What is a reducer in LangGraph state, and when do you need one?
By default a node's update to a state field overwrites that field. A reducer customises the merge — for example, `Annotated[list, add]` appends rather than overwrites. You need a reducer for fields that should accumulate, such as a conversation history or a growing list of retrieved documents.

Q7. What is checkpointing and what does it enable?
Checkpointing saves the graph's full state after every node. It enables durable execution (resuming after a crash or pause), human-in-the-loop (pausing for approval, then resuming), memory (checkpoints grouped by a thread_id form a conversation), and time travel (rewinding to an earlier checkpoint).

Q8. Explain the difference between short-term and long-term agent memory.
Short-term memory is the current conversation's history — in LangGraph, provided by a checkpointer keyed on a thread_id. Long-term memory persists across separate conversations and is stored in a separate store, commonly a vector store the agent writes memories into and retrieves from by similarity — effectively RAG over the agent's own past.

ADVANCED

Q9. Compare the ReAct, plan-and-execute, and reflection patterns.
ReAct loops reason → act → observe, deciding one step at a time; it is flexible and the default for single-agent tool use. Plan-and-execute first produces an explicit multi-step plan, then executes it (with optional re-planning); it keeps complex multi-stage tasks goal-directed. Reflection adds a critic that evaluates the agent's output and feeds back revisions in a loop; it raises quality at the cost of extra LLM calls. They compose — e.g. plan-and-execute at the top, ReAct within steps, reflection on the final answer. Self-RAG is reflection applied to RAG.

Q10. How would you build Corrective RAG with LangGraph?
Define a state holding the question, retrieved documents, the answer, and a retry counter. Create nodes for retrieve, grade-documents, rewrite-query, and generate. Wire START → retrieve → grade-documents, then a conditional edge out of grade-documents: if relevant chunks were found route to generate, otherwise (with retries remaining) route to rewrite-query, which has a normal edge back to retrieve — forming the self-correction loop. The retry counter in the router is the loop guard. Optionally route exhausted retrievals to a web-search fallback node before generating.

Q11. What must be added to take an agent from prototype to production?
Loop guards (retry counters and a recursion limit) so loops always terminate; cost and latency budgeting and monitoring, since agents make many LLM calls; a persistent checkpointer for durable execution and memory; human-in-the-loop approval before consequential actions; observability and tracing, since an agent's path is dynamic and otherwise undebuggable; graceful tool-failure handling so a failed tool reroutes rather than crashes; and guardrails constraining inputs, outputs, and which tools may be called.

SCENARIO-BASED

Q12. Your team deployed an agentic RAG system and received a surprisingly large LLM bill overnight, with some user requests never returning a response. What likely went wrong and how do you fix it?

Almost certainly an unbounded loop. A conditional edge that routes "retry" whenever results are imperfect, with no retry counter and no recursion limit, will loop indefinitely on a hard or unanswerable query — making LLM calls forever, never reaching END, so the request never returns and the cost climbs without bound. The fix: add a retry counter to the state, check it in the routing function so the agent stops retrying after a small number of attempts (and instead generates honestly or falls back), and set a graph-level recursion limit as an independent backstop. Add cost monitoring and alerting so a runaway is caught in minutes, not overnight.

Q13. You are asked to build an assistant that researches a topic across documents and the web, writes a structured report, and checks the report for accuracy before delivering it. How would you architect this with LangGraph?

Use multi-agent orchestration under a supervisor, combining patterns. A supervisor graph routes the task to specialised agents: a researcher agent (a ReAct loop with retrieval and web-search tools) gathers information; a writer agent drafts the structured report; a critic agent applies the reflection pattern, grading the report for faithfulness to the gathered sources and looping revisions back to the writer until satisfied or a retry limit is reached. Each agent is itself a LangGraph subgraph used as a node in the supervisor graph. Add a shared state carrying the research findings and the draft, checkpointing for durability and memory, loop guards on the critic's revision cycle, tracing for observability, and a human-in-the-loop approval step before final delivery.

END OF MODULE 8

You can now build RAG systems that *think* — that decide, branch, loop, self-correct, remember, and coordinate as multiple agents. But a system that reasons is only trustworthy if you can *prove* it works. So far "better" has meant intuition. **Module 9 — RAG Evaluation through RAGAS** makes quality measurable: faithfulness, context precision, context recall, and answer relevancy — the metrics that turn "it seems good" into a number you can track, compare, and defend.